

NIFTY Research Lab — Complete Prototype

AI Trading Research Assistant • ASK → CLARIFY → DEFINE → TEST → LEARN

FILE 1 — app.py

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.graph_objects as go
from datetime import datetime

st.set_page_config(page_title="NIFTY Research Lab", layout="wide")

# HEADER
st.title("Part 1: AI Trading Research Assistant")
st.caption("ASK → CLARIFY → DEFINE → TEST → LEARN")

# 1. ASK
st.header("1. ASK")
user_query = st.text_area(
    "Enter your trading hypothesis:",
    "Does buying NIFTY after a 1% fall work better during high-volatility periods?",
    height=80
)

if st.button("Analyze Hypothesis"):
    st.session_state["analyzed"] = True

if "analyzed" not in st.session_state:
    st.stop()

# 2. CLARIFY
st.header("2. CLARIFY - Ambiguity Detection")
st.warning("Your query has 4 ambiguities. A naive system would assume. We clarify:")

col1, col2 = st.columns(2)

with col1:
    st.subheader("Detected Ambiguities")
    st.markdown("""
    - **1% fall:** vs Previous Close? or Intraday from high?
    - **High-volatility:** VIX > 20? ATR? What threshold?
    - **Work better:** Higher win-rate? Higher avg return?
    - **Holding:** How many days you hold?
    """)

with col2:
    st.subheader("Your Definition (You decide)")
    fall_def = st.selectbox(
        "1% fall definition",
        ["vs Previous Close", "Intraday drawdown from day high"]
    )
    vol_def = st.slider("High Volatility Threshold (VIX proxy)", 15, 30, 20)
    holding = st.slider("Holding Period (days)", 1, 20, 5)
    metric = st.selectbox(
        "Work better means",
        ["Higher Win Rate", "Higher Avg Return", "Higher Sharpe"]
    )

st.info(
    f"""Intelligent Assumption:** I'll assume you mean: "
    f"{fall_def}, VIX > {vol_def}, Hold {holding} days. "
    "I show my assumption, not hide it."
)

# 3. DEFINE
st.header("3. DEFINE - Structured Experiment")

experiment = {
    "Instrument": "NIFTY 50",
    "Entry Rule": f"NIFTY closes {fall_def} < -1%",
    "Filter": f"Volatility (VIX proxy) > {vol_def}",
    "Exit Rule": f"Exit after {holding} trading days",
    "Holding": f"{holding} days",
    "Success Metric": metric,
    "Test Period": "Last 5 Years (2021-2026)",
    "Costs": "0.1% brokerage + slippage per trade",
    "Data Source": "Mock NIFTY data (Yahoo Finance logic)"
}

st.json(experiment)

# 4. TEST
st.header("4. TEST - Backtest")

@st.cache_data
```

```

def generate_mock_data():
    np.random.seed(42)
    dates = pd.date_range(end=datetime.now(), periods=1250)
    returns = np.random.normal(0.0004, 0.012, 1250)

    # Inject high-volatility regimes
    returns[200:300] = np.random.normal(0, 0.025, 100)

    price = 15000 * np.exp(np.cumsum(returns))

    vix_proxy = (
        pd.Series(np.random.normal(18, 5, 1250))
        .rolling(20)
        .mean()
        .values
        + np.abs(returns) * 500
    )

    df = pd.DataFrame({
        "Date": dates,
        "Close": price,
        "VIX_proxy": vix_proxy
    })
    df["Daily_Ret"] = df["Close"].pct_change()
    return df

df = generate_mock_data()

# Find entry signals
df["Signal"] = (
    (df["Daily_Ret"] <= -0.01)
    & (df["VIX_proxy"] > vol_def)
)

trades = []

for i in df[df["Signal"]].index:
    if i + holding < len(df):
        entry = df.loc[i, "Close"]
        exit_p = df.loc[i + holding, "Close"]
        ret = (exit_p - entry) / entry - 0.001
        trades.append(ret)

trades = pd.Series(trades)

col_a, col_b, col_c = st.columns(3)

col_a.metric("Total Trades Found", len(trades))
col_b.metric(
    "Win Rate",
    f"{{(trades > 0).mean() * 100:.1f}}%"
    if len(trades) > 0 else "0%"
)
col_c.metric(
    "Avg Return / Trade",
    f"{{trades.mean() * 100:.2f}}%"
    if len(trades) > 0 else "0%"
)

fig = go.Figure()

fig.add_trace(
    go.Scatter(
        x=df["Date"],
        y=df["Close"],
        name="NIFTY"
    )
)

fig.add_trace(
    go.Scatter(
        x=df[df["Signal"]]["Date"],
        y=df[df["Signal"]]["Close"],
        mode="markers",
        name="Entry (1% fall + High Vol)",
        marker=dict(size=10, color="red")
    )
)

st.plotly_chart(fig, use_container_width=True)

# 5. LEARN
st.header("5. LEARN - What We Can Actually Conclude")

st.subheader("A) What Data ACTUALLY Shows vs What System BELIEVES")

c1, c2 = st.columns(2)

with c1:
    st.success("***Data Shows (Fact):**")
    st.write(

```

```

        f"- In {len(trades)} such events, win rate is "
        f"{(trades > 0).mean() * 100:.1f}%"
    )
    st.write(
        f"- Avg return is {trades.mean() * 100:.2f}% after costs"
        if len(trades) > 0 else "- Avg return cannot be calculated"
    )
    st.write("- Sample size may be small during high-volatility periods")

with c2:
    st.error("***System Believes (Interpretation, can be wrong):**")
    st.write("- It might work slightly better in high volatility, but confidence is LOW")
    st.write("- A difference should be tested for statistical significance")
    st.write("- The apparent edge could be noise rather than a real advantage")

st.subheader("B) What Could Go Wrong - Risks")

st.markdown("""
- **Look-ahead bias:** Are we using VIX information known at the entry time?
- **Overfitting:** Was the volatility threshold selected after seeing results?
- **Transaction cost & Slippage:** Real execution costs can increase during high volatility.
- **Survivorship bias:** The selected period may not represent all market regimes.
- **No stop-loss tested:** Downside risk is not fully captured.
""")

st.subheader("C) What I Would Improve Next")

st.markdown("""
1. Test with real NSE bhavcopy data + actual India VIX.
2. Compare against a baseline: buy after a 1% fall in low volatility.
3. Add a statistical significance test.
4. Add stop-loss and position sizing.
""")

```

FILE 2 — requirements.txt

```
streamlit
pandas
numpy
plotly
```

FILE 3 — README.md

```
# NIFTY Research Lab
```

```
## Architecture
```

```
Single-page Streamlit app with no separate backend.
```

```
## Technology
```

- Streamlit for the interactive interface
- Plotly for visualization
- Pandas and NumPy for the backtest logic

```
## Key Assumption
```

```
Mock data is used to demonstrate the research workflow and logic. It is **not real NIFTY market data and does not represent real trading.
```

```
## Research Flow
```

```
ASK → CLARIFY → DEFINE → TEST → LEARN
```

```
The CLARIFY step is the central product-thinking feature. Instead of silently making assumptions, the application exposes ambiguity and asks for clarification.
```

```
## How to Run
```

```
```bash
pip install -r requirements.txt
streamlit run app.py
```
```

```
## Important Note
```

```
This prototype is for research/educational demonstration only and should not be treated as financial advice or a live trading system.
```

AI Usage & Thinking Note

AI Usage & Thinking Note

1. Product Decision

The main design decision was to make **CLARIFY** the hero section rather than simply showing a backtest result. A trading hypothesis can contain ambiguous terms such as “1% fall,” “high volatility,” “works better,” and “holding period.”

The prototype therefore asks the user to define these terms before testing.

2. AI Usage

Tools Used: ChatGPT for parsing logic structure and Muse for Plotly chart code.

Used for: Boilerplate backtest loop, UI layout, and code structuring.

3. Decisions I Made Myself

I decided to make CLARIFY the hero section instead of TEST. I rejected the idea of directly showing a backtest result without first defining the hypothesis.

I also added an explicit separation between:

- **Data Shows (Fact)**
- **System Believes (Interpretation)**

This makes uncertainty visible and avoids presenting a model interpretation as if it were a proven fact.

4. What I Rejected

I rejected the suggestion to use an LSTM or another sophisticated prediction model because the assignment emphasizes thinking and product reasoning rather than model complexity.

A more complex model would not solve the underlying ambiguity in the user's hypothesis.

5. Risks Considered

The prototype explicitly highlights possible problems such as look-ahead bias, overfitting, transaction costs and slippage, survivorship bias, and incomplete downside-risk testing.

6. What I Would Improve Next

The next version should use real NSE data and actual India VIX data. It should also include a low-volatility baseline, statistical significance testing, stop-loss rules, and position sizing.

7. Key Learning

The most important learning is that a research assistant should not simply produce a number. It should help the user define what they actually want to test, show assumptions clearly, distinguish evidence from interpretation, and explain what could make the conclusion unreliable.

